

spGPU: Architecture and System Specification

Design, Implementation, and Evaluation of a 2D/2.5D Multicore Tiled Graphics Processing Unit on FPGA SoC

Digital Signal Processing (DSP) Course Project
Sapienza University of Rome

Luca Filogna & Valerio Cilento

*Faculty of Information Engineering, Computer Science and Statistics
Department of Information Engineering, Electronics and Telecommunications (DIET)
Sapienza University of Rome, Italy*

Academic Year 2025–2026

Revision 2.1 – Full System Documentation

Abstract

This report provides a comprehensive architectural and engineering specification of **spGPU**, a custom, high-performance 2D/2.5D multicore Graphics Processing Unit implemented on a Xilinx Zynq-7000 All-Programmable System-on-Chip (SoC) hosted on the Digilent PYNQ-Z1 development board.

spGPU adopts a spatially partitioned, tile-based architecture comprising ten independent compute cores (**spCORE**) executing in parallel at 100.0 MHz. Each core is tightly coupled to a dedicated on-chip True Dual-Port Block RAM tile (320×24 pixels), providing a total rendering resolution of 320×240 with 15-bit RGB555 color depth. Dedicated integer hardware rasterization engines accelerate geometric primitives, including Bresenham lines, midpoint circles, solid disc scanlines, and edge-equation triangle fills.

An integrated 4-bit depth buffer (Z-buffer) is implemented in every tile memory, executing single-cycle depth comparisons to resolve occlusion and 2.5D layering autonomously. A dynamic instruction scheduler calculates primitive bounding boxes in combinational logic, streaming 64-bit spISA commands exclusively to the overlapping tile FIFOs to maximize compute efficiency. The video subsystem provides $2\times$ hardware nearest-neighbor upscaling, TMDS 8b/10b encoding, and 5:1 DDR serialization at 125.0 MHz via **USERDESE2** primitives, driving an HDMI display at 640×480 @ 60 Hz with zero tearing through VSYNC-locked ping-pong double buffering.

A dual-interrupt synchronization architecture connects the GPU to an ARM Cortex-A9 host running a bare-metal Newtonian physics engine, demonstrating rock-solid 60 FPS performance verified through desktop visual co-simulation and hardware synthesis.

Contents

1	Introduction and Project Overview	6
1.1	Academic Context and Project Scope	6
1.2	Target Hardware Platform	6
1.3	Key Architectural Specifications	7
1.4	Design Methodology and Toolflow	7
2	System-on-Chip (SoC) Architecture and Interconnect	8
2.1	System Partitioning: Processing System vs. Programmable Logic	8
2.2	AXI DMA Communication Subsystem	8
2.3	AXI4-Stream Custom Peripheral (<code>myaxistream_1_0</code>)	10
2.3.1	State Machine and Handshaking	10
3	Instruction Set Architecture (spISA)	11
3.1	Instruction Set Overview	11
3.2	Bitfield Definitions and Allocations	11
3.3	Detailed Instruction Specifications	12
3.3.1	Broadcast Instructions vs. Spatial Primitives	12
4	Multicore Tiling and Spatial Parallelism	14
4.1	Spatial Decomposition Methodology	14
4.2	Core-to-Tile Mapping and Coordinate Spaces	15
4.3	Throughput Scaling and Hardware Benefits	15
5	Dynamic Instruction Scheduler	16
5.1	Role and Motivation of the Scheduler	16
5.2	Combinatorial Bounding Box Pre-Decoder	16
5.3	Core Range Classification and Selective Dispatching	17
5.4	Dedicated Synchronous FIFOs and Flow Control	17
5.4.1	Backpressure Mechanism	18
6	Single-Core Microarchitecture (spCORE)	19
6.1	Structural Overview of the Compute Core	19
6.2	Pipeline Control and Decode Unit (<code>spPIPE.vhd</code>)	19
6.2.1	Pipeline Finite State Machine	20
6.2.2	Parameter Registration	20
6.3	Execution Unit and Tile Boundary Protection (<code>spEXEC.vhd</code>)	20
6.3.1	Candidate Pixel Multiplexing and Completion Aggregation	20
6.3.2	Hardware Tile Clipping Unit	21

7	Hardware Primitive Accelerators	22
7.1	Overview of Dedicated Acceleration Engines	22
7.2	Direct Pixel Writer (<code>DRAWPIXEL</code>)	22
7.3	Bresenham Line Accelerator (<code>line_acc.vhd</code>)	23
7.3.1	Mathematical Formulation and Data Path	23
7.4	Midpoint Circle Accelerator (<code>circle_acc.vhd</code>)	23
7.4.1	Eight-Octant Traversal	23
7.5	Filled Circle Accelerator (<code>filled_circle_acc.vhd</code>)	23
7.6	Filled Triangle Rasterizer (<code>edge_fill_top_v2.vhd</code>)	24
7.6.1	Bounding Box Determination (<code>minmax.vhd</code>)	24
7.6.2	Parallel Edge Function Evaluation (<code>edge_fill_v2.vhd</code>)	24
7.6.3	Half-Space Inclusion Test	24
8	Tile Framebuffer, Double Buffering, and Hardware Z-Buffer	25
8.1	Memory Architecture Overview	25
8.2	True Dual-Port BRAM Organization and Double Buffering	25
8.3	Swap Protocol and Hardware Background Auto-Clear	26
8.4	Integrated 4-Bit Hardware Z-Buffer (Depth Test)	26
8.4.1	Single-Cycle Depth Comparator	26
8.5	Multiplier-Free Address Generation	27
9	Display Controller and Physical HDMI Transmitter	28
9.1	Video Subsystem Overview	28
9.2	VGA Timing Generation (<code>counter_pixel.vhd</code>)	28
9.3	Hardware 2× Nearest-Neighbor Upscaling	28
9.4	BRAM Pipeline Latency Alignment	30
9.5	Color Expansion (RGB555 to RGB888)	30
9.6	TMDS 8b/10b Encoders (<code>tmds_encoder.vhd</code>)	30
9.7	Physical Serialization and Differential I/O (<code>serializer.vhd</code>)	31
10	Performance Analyzer and Dual-Interrupt Architecture	32
10.1	Hardware Telemetry Subsystem (<code>spANALYZER.vhd</code>)	32
10.2	Dual-Interrupt Hardware Architecture	32
10.3	Clock Domain Crossing for VSYNC Pulse Generation	32
10.4	Frame Pacing and Zero-Tearing Lock	34
11	Bare-Metal Software Stack and Host Driver	35
11.1	Software Architecture Overview	35
11.2	Low-Level AXI DMA Driver	35
11.3	Zero-Copy Bit-Packed Instruction Mapping	36
11.4	High-Level Drawing API	36
11.5	Real-Time Physics Demonstration Application (<code>main.c</code>)	37
11.5.1	Physical Model and Collision Resolution	37
11.5.2	Execution Loop and Console Output	37
12	Simulation, Verification, and Tooling	38
12.1	Verification Strategy	38
12.2	Software Simulation Models (<code>software_sim/</code>)	38
12.3	Interactive Visual Simulator (<code>simulator.c</code>)	39
12.4	RTL Testbench Suite and ModelSim Automation	39

13 Implementation Results and Design Discussion	40
13.1 FPGA Synthesis and Resource Utilization	40
13.2 Clock Domain Architecture and Timing Closure	40
13.3 Analysis of Architectural Innovations and Trade-Offs	41
13.4 Potential Future Enhancements	41
14 Conclusion	42

List of Figures

2.1	spGPU System-on-Chip (SoC) Top-Level Architecture and Interconnect.	9
3.1	spISA 64-Bit Instruction Word Bitfield Allocations.	11
4.1	spGPU Multicore Spatial Tiling and Display Scanning Architecture.	14
5.1	Dynamic Instruction Scheduler (spScheduler) Microarchitecture.	16
6.1	Microarchitecture of the Single Graphics Core (spCORE).	19
7.1	Mathematical Data Paths of the Hardware Accelerators: (a) Bresenham Line Unit, (b) Triangle Edge Equations Rasterizer, and (c) Filled Circle Scanline Generator.	22
8.1	Microarchitecture of the Tile Framebuffer and Integrated Z-Buffer (tile.vhd). .	26
9.1	Complete HDMI Display and Physical Transmission Pipeline.	29
10.1	Dual-Interrupt Hardware/Software Synchronization and Frame Timing Diagram.	33

List of Tables

1.1	spGPU Hardware Architectural Specifications	7
3.1	spISA 64-Bit Instruction Set Summary	12
4.1	Multicore Tile Mapping and Memory Footprint	15
10.1	Hardware Interrupt Mapping to ARM Cortex-A9 GIC	33
13.1	Estimated FPGA Resource Utilization on Xilinx Zynq XC7Z020	40

Chapter 1

Introduction and Project Overview

1.1 Academic Context and Project Scope

The **spGPU** (*Sapienza Graphics Processing Unit*) project was conceived, designed, and implemented as an advanced digital design and hardware acceleration project for the *Digital Signal Processing (DSP)* course at **Sapienza University of Rome**, authored by **Luca Filogna** and **Valerio Cilento**.

The primary objective of the project is the full-stack design, algorithmic prototyping, Register-Transfer Level (RTL) implementation in VHDL, software simulation, and physical FPGA synthesis of a custom **2D/2.5D Multicore Graphics Processing Unit**. Rather than relying on commercial graphics IP cores or software-driven framebuffers, spGPU represents an entirely custom hardware architecture featuring parallel compute pipelines, distributed on-chip video memory, hardware rasterizers, an integrated depth buffer (Z-buffer), and a direct High-Definition Multimedia Interface (HDMI) physical transmitter.

1.2 Target Hardware Platform

The complete system is synthesized and validated on the **Digilent PYNQ-Z1** development board, built around the **Xilinx Zynq-7000 All-Programmable SoC (xc7z020-1clg400c)**. The architecture takes full advantage of the heterogeneous SoC paradigm:

- **Processing System (PS)**: Hosts a Dual-Core ARM Cortex-A9 MPCore microprocessor operating up to 667 MHz, responsible for running the application game loop, rigid-body physics simulation, collision detection, and issuing drawing commands.
- **Programmable Logic (PL)**: Implements the spGPU multicore processor, the instruction dispatcher, the distributed dual-port Block RAM framebuffers, the hardware Z-buffer, the hardware performance monitor, and the HDMI physical layer.
- **On-Chip Interconnect**: High-throughput AXI4 interfaces and an AXI Direct Memory Access (DMA) engine in Simple Mode that streams graphics instructions directly from the ARM memory space into the GPU instruction scheduler.

1.3 Key Architectural Specifications

Table 1.1 summarizes the fundamental architectural parameters and performance metrics of the spGPU system.

Table 1.1: spGPU Hardware Architectural Specifications

Parameter	Specification / Value
Target Device	Xilinx Zynq-7000 SoC (xc7z020c1g400-1)
Development Board	Digilent PYNQ-Z1
Synthesis Toolflow	AMD Xilinx Vivado 2023.1
Host Processor (PS)	Dual-Core ARM Cortex-A9 (Bare-Metal C)
GPU Clock Domain	100.0 MHz (Core compute, memory write, scheduler)
Pixel Clock Domain	25.0 MHz (Display scanning, VGA timing generator)
Serial Clock Domain	125.0 MHz (5× pixel clock, TMDS 5:1 DDR serialization)
Logical Rendering Resolution	320 × 240 pixels
Physical HDMI Output	640 × 480 @ 60 Hz (Standard 480p VGA timing)
Hardware Spatial Upscaler	2× Horizontal & 2× Vertical Nearest-Neighbor
Color Representation	15-bit RGB555 (32,768 colors), expanded to 24-bit RGB888
Depth Buffer (Z-Buffer)	Integrated 4-bit per pixel (16 discrete depth layers)
Spatial Parallelism	10 Independent Compute Cores (spCORE)
Tiling Geometry	10 Horizontal Tiles of 320 × 24 pixels each
On-Chip VRAM Structure	10 Distributed True Dual-Port Block RAM tiles
Buffering Scheme	Double-Buffering (Ping-Pong VRAM) with VSYNC-lock
Background Auto-Clear	Hardware zero-overhead back-buffer clearing upon swap
Communication Protocol	AXI DMA (Memory-Mapped to Stream) + 64-bit spISA
Hardware Telemetry	Integrated 100 MHz FPS counter with dual GIC interrupts

1.4 Design Methodology and Toolflow

The engineering lifecycle followed for spGPU adopted a rigorous, verification-driven methodology:

1. **Software Prototyping & Golden Models:** High-level C/C++ algorithms were formulated for all primitive rasterization techniques (Bresenham lines, midpoint circles, scanline spans, and edge-equation half-space triangle filling) and OpenCV-based nearest-neighbor scaling.
2. **Visual RTL Co-Simulation:** A desktop graphical simulator was constructed using the *Raylib* library, capable of consuming pixel dump files generated during VHDL testbench simulations and rendering the exact visual frame to verify circuit correctness before FPGA implementation.
3. **RTL Synthesis and Place-and-Route:** Synthesized using AMD Vivado 2023.1, enforcing multi-clock domain constraints (100 MHz core clock, 25 MHz pixel clock, 125 MHz serial clock) and optimizing logic placement across the 28nm Artix-7 fabric.
4. **Bare-Metal Integration:** Construction of memory-mapped peripheral drivers, interrupt service routines (ISRs) via the ARM Generic Interrupt Controller (GIC), and deployment via bootable SD card images (BOOT.bin).

Chapter 2

System-on-Chip (SoC) Architecture and Interconnect

2.1 System Partitioning: Processing System vs. Programmable Logic

The spGPU architecture adopts a hardware/software co-design strategy that leverages the Xilinx Zynq-7000 All-Programmable System-on-Chip (SoC). The system is physically partitioned into two tightly coupled domains:

1. **Processing System (PS):** Hosts the dual-core ARM Cortex-A9 processor running bare-metal C firmware. The CPU executes high-level tasks that require complex branching, floating-point calculations, and dynamic state evaluation, including rigid-body physics, ball-to-ball and boundary elastic collision resolution, and frame timing orchestration.
2. **Programmable Logic (PL):** Implements the spGPU coprocessor, containing the instruction dispatching unit, the parallel compute cores, the distributed tile memories, the performance analyzer, and the physical HDMI output engine.

Figure 2.1 illustrates the global block diagram of the SoC architecture, highlighting the interconnect interfaces, clock domains, and interrupt lines connecting the ARM processor with the FPGA logic.

2.2 AXI DMA Communication Subsystem

To offload graphics commands without burdening the CPU with per-pixel or per-word Programmed I/O (PIO) transactions, communication from the PS to the PL is mediated by the **Xilinx AXI Direct Memory Access (AXI DMA)** core configured in *Simple Direct Register Mode* for Memory-Mapped to Stream (MM2S) transfers:

- **High-Speed Memory Read:** The DMA controller accesses DDR memory over the 32-bit AXI High-Performance (HP) slave port on the PS.
- **Stream Conversion:** Instructions formatted into contiguous memory buffers are streamed over a 64-bit AXI4-Stream master interface (`M_AXIS_MM2S`).

- **Low Software Overhead:** The CPU merely writes the source address (`MM2S_SA`) and transfer byte length (`MM2S_LENGTH`) to the DMA memory-mapped control registers, allowing the hardware engine to burst instructions autonomously.

2.3 AXI4-Stream Custom Peripheral (`myaxistream_1_0`)

The raw AXI4-Stream interface from the DMA engine is received on the FPGA by a custom conversion peripheral, `myaxistream_1_0` (detailed in `myaxistream_v1_0_S_AXIS.vhd`). This module serves as the primary gateway and protocol adapter between the Xilinx AXI bus and the internal scheduler of spGPU.

2.3.1 State Machine and Handshaking

The stream interface operates under a three-state Finite State Machine (FSM):

1. **WAIT_S:** The receiver remains in idle until the GPU scheduler asserts `ready_enb` (indicating that the core FIFOs are ready to absorb a new command). Upon detecting a rising edge on `ready_enb`, the FSM transitions to `TX1_S`.
2. **TX1_S:** The core asserts `S_AXIS_TREADY <= '1'`, signalling to the DMA master that it is prepared to accept data. When `S_AXIS_TVALID` is sampled high, the FSM advances to `TX2_S`.
3. **TX2_S:** The 64-bit data bus `S_AXIS_TDATA` is latched into `instr_word`, `instr_valid` is pulsed high for one cycle, `S_AXIS_TREADY` is deasserted, and the FSM returns to `WAIT_S`.

Listing 2.1 shows the core VHDL FSM implementation of this handshake protocol.

```

1 process(all)
2 begin
3     if S_AXIS_ARESETN = '0' then
4         next_state_t <= WAIT_S;
5     else
6         case state_t is
7             when WAIT_S =>
8                 if fsm_start_edge = '1' then
9                     next_state_t <= TX1_S;
10                else
11                    next_state_t <= WAIT_S;
12                end if;
13             when TX1_S =>
14                 if S_AXIS_TVALID = '1' then
15                     next_state_t <= TX2_S;
16                 else
17                     next_state_t <= TX1_S;
18                 end if;
19             when TX2_S =>
20                 next_state_t <= WAIT_S;
21             when others =>
22                 next_state_t <= WAIT_S;
23         end case;
24     end if;
25 end process;

```

Listing 2.1: AXI4-Stream Handshake and State Machine in `myaxistream_v1_0_S_AXIS.vhd`.

This handshake guarantees that incoming 64-bit instruction words from the DMA engine are ingested synchronously into the GPU clock domain without risk of FIFO overflow or lost words.

Chapter 3

Instruction Set Architecture (spISA)

3.1 Instruction Set Overview

The spGPU architecture defines a dedicated, fixed-length 64-bit instruction set architecture designated as **spISA**. The fixed 64-bit format was deliberately chosen to maximize data throughput over standard AXI interconnects, simplify hardware decode logic, and eliminate instruction alignment penalties.

Each instruction encapsulates the operation code (**Opcode**), optional depth metadata (Z), geometric vertex coordinates, radius, or color attributes in a single packed data word.

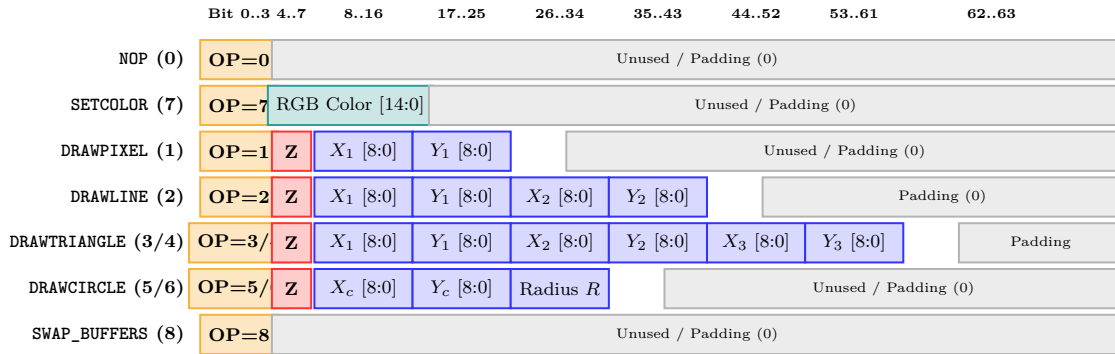


Figure 3.1: spISA 64-Bit Instruction Word Bitfield Allocations.

3.2 Bitfield Definitions and Allocations

As defined in `sp-pkg.vhd` and illustrated in Figure 3.1, the 64-bit instruction fields are structured as follows:

- **Opcode (OP):** Bits [3:0] specify the primary operation code. An 8-bit allocation ($N_{opcode} = 8$) is reserved in the hardware package to support future ISA expansions.
- **Depth Coordinate (Z):** Bits [7:4] define a 4-bit unsigned depth value for geometric drawing instructions, providing 16 discrete depth sorting layers ($0 \dots 15$). A value of $Z = 0$

represents the nearest layer to the viewer, while $Z = 15$ represents the farthest background plane.

- **Pixel Coordinates (X, Y):** Discrete spatial coordinates are encoded as 9-bit unsigned integer fields (`N_pixel = 9`), providing an addressable coordinate space of $[0, 511]$, which naturally covers the 320×240 screen geometry.
- **Color Value (`Color`):** Encoded as a 15-bit RGB555 integer within bits $[22:8]$ of the `SETCOLOR` instruction (5 bits Red, 5 bits Green, 5 bits Blue), capable of expressing 32,768 distinct colors.

3.3 Detailed Instruction Specifications

Table 3.1 summarizes the complete set of supported operations in spISA.

Table 3.1: spISA 64-Bit Instruction Set Summary			
Opcode	Binary	Mnemonic	Functional Description and Parameters
0	0000	NOP	No Operation. Propagated in broadcast to all cores.
1	0001	DRAWPIXEL	Renders a single pixel at (X_1, Y_1) with depth Z .
2	0010	DRAWLINE	Renders a 1-pixel wide line segment between (X_1, Y_1) and (X_2, Y_2) with depth Z .
3	0011	DRAWTRIANGLE	Draws the outline of a triangle with vertices (X_1, Y_1) , (X_2, Y_2) , and (X_3, Y_3) with depth Z .
4	0100	DRAWTRIANGLE_F	Rasterizes a solid filled triangle with vertices (X_1, Y_1) , (X_2, Y_2) , and (X_3, Y_3) with depth Z .
5	0101	DRAWCIRCLE	Draws the outline of a circle centered at (X_c, Y_c) with radius R and depth Z .
6	0110	DRAWCIRCLE_F	Rasterizes a solid filled disk centered at (X_c, Y_c) with radius R and depth Z .
7	0111	SETCOLOR	Updates the active 15-bit color register in all cores to the RGB555 value encoded in bits $[22 : 8]$.
8	1000	SWAP_BUFFERS	Global synchronization instruction. Triggers front/back buffer swap across all 10 tiles upon VSYNC.

3.3.1 Broadcast Instructions vs. Spatial Primitives

Instructions in spISA are categorized into two fundamental operational classes:

1. **Global Broadcast Instructions:** NOP (0), SETCOLOR (7), and SWAP_BUFFERS (8). These instructions modify the global state of the rendering pipeline. Consequently, the hardware scheduler broadcasts these words unconditionally into the command FIFOs of all 10 compute cores simultaneously.

2. **Spatial Drawing Primitives:** DRAWPIXEL (1), DRAWLINE (2), DRAWTRIANGLE (3), DRAWTRIANGLE_F (4), DRAWCIRCLE (5), and DRAWCIRCLE_F (6). These primitives describe localized 2D shapes. The scheduler computes their vertical bounding boxes and dispatches the instruction only to the specific cores whose screen tiles intersect the primitive, as elaborated in Chapter 5.

Chapter 4

Multicore Tiling and Spatial Parallelism

4.1 Spatial Decomposition Methodology

A primary challenge in high-performance GPU design is resolving the "memory wall"—the bottleneck where multiple parallel compute pipelines compete for read and write bandwidth to a centralized frame buffer.

To completely eliminate memory arbitration delays and bus contention, spGPU implements a **Tile-Based Spatial Partitioning** architecture. The full rendering canvas of 320×240 pixels is subdivided horizontally into 10 discrete, non-overlapping strips called *Tiles*:

$$H_{\text{total}} = 240 \text{ lines}, \quad N_{\text{cores}} = 10 \implies H_{\text{tile}} = \frac{H_{\text{total}}}{N_{\text{cores}}} = \frac{240}{10} = 24 \text{ lines/tile} \quad (4.1)$$

Each tile encompasses an area of $320 \times 24 = 7,680$ pixels. A dedicated compute core (spCORE) and a dedicated, isolated dual-port Block RAM memory block (tile.vhd) are assigned to each tile, as illustrated in Figure 4.1.

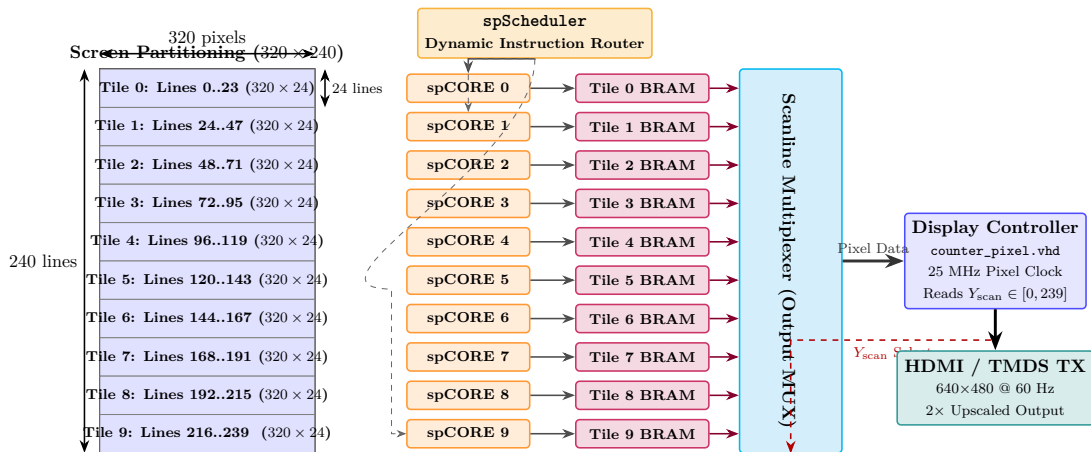


Figure 4.1: spGPU Multicore Spatial Tiling and Display Scanning Architecture.

4.2 Core-to-Tile Mapping and Coordinate Spaces

Each core $i \in [0, 9]$ operates on global coordinates during instruction decode, but writes exclusively into its local 320×24 tile VRAM. The vertical start offset of each tile is precomputed at compile time by the deferred constant function `calc_y_tile_start` in `sp-pkg.vhd`:

$$Y_{\text{tile_start}}(i) = i \times 24 \quad (4.2)$$

Table 4.1 specifies the physical mapping between cores, tiles, scanlines, and memory allocations.

Table 4.1: Multicore Tile Mapping and Memory Footprint

Core ID	Assigned Tile	Global Y Scanlines	Local Y Range	VRAM Footprint (2× Buffer)
spCORE 0	Tile 0	0...23	0...23	15,360 × 15-bit (230.4 kb)
spCORE 1	Tile 1	24...47	0...23	15,360 × 15-bit (230.4 kb)
spCORE 2	Tile 2	48...71	0...23	15,360 × 15-bit (230.4 kb)
spCORE 3	Tile 3	72...95	0...23	15,360 × 15-bit (230.4 kb)
spCORE 4	Tile 4	96...119	0...23	15,360 × 15-bit (230.4 kb)
spCORE 5	Tile 5	120...143	0...23	15,360 × 15-bit (230.4 kb)
spCORE 6	Tile 6	144...167	0...23	15,360 × 15-bit (230.4 kb)
spCORE 7	Tile 7	168...191	0...23	15,360 × 15-bit (230.4 kb)
spCORE 8	Tile 8	192...215	0...23	15,360 × 15-bit (230.4 kb)
spCORE 9	Tile 9	216...239	0...23	15,360 × 15-bit (230.4 kb)
Total	10 Tiles	0 ... 239	—	153,600 words (≈ 2.30 Mb)

4.3 Throughput Scaling and Hardware Benefits

The architectural benefits of this spatial partitioning are significant:

1. **True Parallel Memory Writes:** At any given clock cycle, all 10 cores can execute write operations simultaneously into their respective BRAM blocks. There are no shared arbiter stalls, crossbar multiplexers, or wait states on the write ports.
2. **Near-Linear Acceleration for Distributed Scenes:** When drawing objects that span multiple vertical regions of the display (such as the 10 bouncing spheres in the demo application), all 10 cores compute concurrently, yielding up to a 10× speedup compared to a single-core implementation.
3. **Decoupled Display Readout:** The display controller reads sequential scanlines for video generation using Port B of the dual-port RAMs. Because each tile is an independent memory primitive, the video controller reads from only one tile at a time via the output multiplexer, while all 10 cores continue writing freely to their respective Port A interfaces.

Dynamic Instruction Scheduler

5.1 Role and Motivation of the Scheduler

In a tiled multicore graphics architecture, broadcasting every drawing instruction unconditionally to every core would force all cores to compute all primitives, defeating the purpose of spatial parallelism and wasting power and clock cycles.

The **spScheduler** component acts as an intelligent hardware router. It receives the incoming stream of 64-bit instructions from the AXI-Stream interface, evaluates the geometric vertical boundaries of the target primitive in single-cycle combinatorial logic, and selectively dispatches the instruction word *only* to the FIFOs of the cores whose tile areas physically intersect the shape.

Figure 5.1 depicts the internal block diagram of the dynamic scheduler, including the pre-decode bounding box logic, the range classifier, the FIFO bank, and the backpressure flow control.

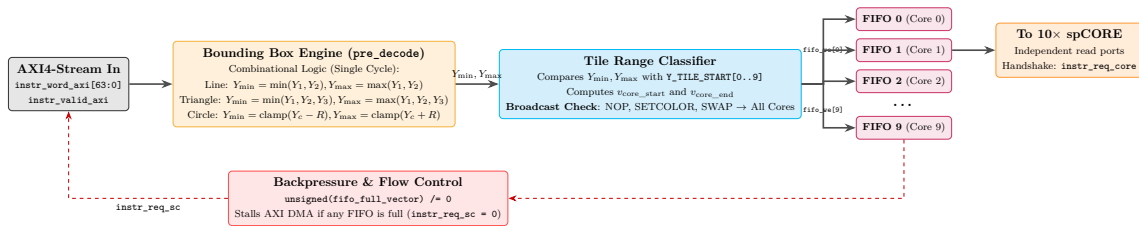


Figure 5.1: Dynamic Instruction Scheduler (**spScheduler**) Microarchitecture.

5.2 Combinatorial Bounding Box Pre-Decoder

During the **request** state, the process **pre_decode** inspects the incoming opcode and vertex coordinates in purely combinatorial logic to determine the vertical interval $[Y_{\min}, Y_{\max}]$:

- **Pixel (DRAWPIXEL):**

$$Y_{\min} = Y_1, \quad Y_{\max} = Y_1 \quad (5.1)$$

- **Line (DRAWLINE):**

$$Y_{\min} = \min(Y_1, Y_2), \quad Y_{\max} = \max(Y_1, Y_2) \quad (5.2)$$

- **Triangles (DRAWTRIANGLE and DRAWTRIANGLE_F):**

$$Y_{\min} = \min(Y_1, \min(Y_2, Y_3)), \quad Y_{\max} = \max(Y_1, \max(Y_2, Y_3)) \quad (5.3)$$

- **Circles (DRAWCIRCLE and DRAWCIRCLE_F):** Because circular geometry can extend partially off-screen, the bounding box evaluation incorporates explicit hardware clamping to prevent unsigned underflow or overflow beyond the screen height (240):

$$Y_{\max} = \begin{cases} 239 & \text{if } Y_c > (240 - R) \\ Y_c + R & \text{otherwise} \end{cases} \quad (5.4)$$

$$Y_{\min} = \begin{cases} 0 & \text{if } Y_c < R \\ Y_c - R & \text{otherwise} \end{cases} \quad (5.5)$$

5.3 Core Range Classification and Selective Dispatching

Once $[Y_{\min}, Y_{\max}]$ is computed, the scheduler evaluates the tile interval $[v_{\text{core_start}}, v_{\text{core_end}}]$ by comparing the bounds against the constant array `Y_TILE_START`:

```

1 v_core_start := 0;
2 v_core_end   := 0;
3
4 for i in 0 to N_cores-1 loop
5   if unsigned(y_min) >= Y_TILE_START(i) then
6     v_core_start := i;
7   end if;
8   if unsigned(y_max) >= Y_TILE_START(i) then
9     v_core_end := i;
10  end if;
11 end loop;
12
13 if v_core_start > v_core_end then
14   v_core_end := v_core_start;
15 end if;
```

Listing 5.1: Core Range Classification Logic in `spScheduler.vhd`.

Based on this range, the scheduler directs writes to the command FIFOs:

- **Broadcast Commands (NOP, SETCOLOR, SWAP_BUFFERS):** The write enable signal `fifo_we_vector(i)` is asserted for **all 10 FIFOs**.
- **Drawing Primitives:** The write enable is asserted **only for cores** $i \in [v_{\text{core_start}}, v_{\text{core_end}}]$. All other FIFOs receive write enable '0'.

5.4 Dedicated Synchronous FIFOs and Flow Control

Each core is backed by an independent synchronous FIFO (`sc_fifo.vhd`) with a parameterized depth (`FIFO_DEPTH = 12 to 32`). These queues buffer instructions, allowing cores that finish small primitives rapidly to proceed without waiting for cores engaged in rasterizing large triangles or filled circles.

5.4.1 Backpressure Mechanism

To guarantee that fast bursts of DMA transfers never overwrite unread FIFO entries, the scheduler aggregates the full status of all 10 FIFOs:

$$\text{instr_req_sc} = \begin{cases} '1' & \text{if } (\text{state} = \text{request}) \wedge \left(\sum_{i=0}^9 \text{fifo_full}(i) = 0 \right) \\ '0' & \text{otherwise} \end{cases} \quad (5.6)$$

If any FIFO in the system becomes full, the scheduler immediately deasserts `instr_req_sc`, causing the upstream AXI4-Stream interface to lower `s_axis_tready`, effectively pausing the DMA transfer until queue space becomes available.

Chapter 6

Single-Core Microarchitecture (spCORE)

6.1 Structural Overview of the Compute Core

Each of the ten identical graphics processing units (**spCORE**) instantiated in the top-level entity is designed as a modular, decoupled processing engine. As shown in Figure 6.1, **spCORE** is internally divided into two primary sub-blocks:

1. **spPIPE** (*Instruction Pipeline, Fetch, Decode & Registers*): Manages the core command interface, decodes 64-bit spISA words, latches vertex and color parameters into dedicated registers, and orchestrates the core execution state machine.
2. **spEXEC** (*Execution Unit, Dedicated Accelerators & Tile Boundary Protection*): Houses the parallel hardware rasterization engines, arbitrates the generated pixel streams, handles swap handshaking, and enforces hard spatial clipping boundaries for the tile.

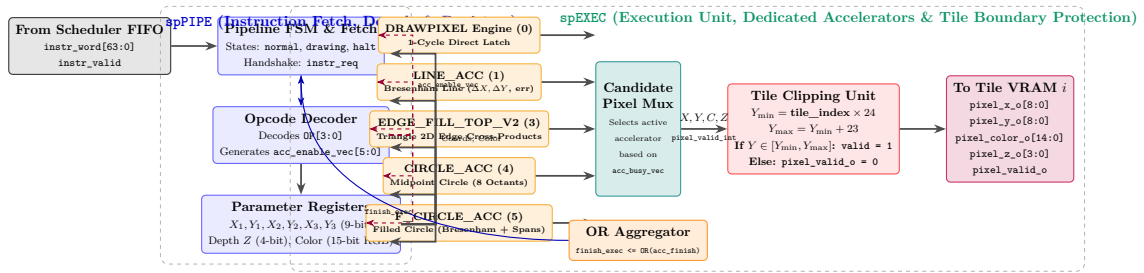


Figure 6.1: Microarchitecture of the Single Graphics Core (spCORE).

6.2 Pipeline Control and Decode Unit (spPIPE.vhd)

The **spPIPE** block is responsible for synchronizing instruction ingestion with accelerator availability.

6.2.1 Pipeline Finite State Machine

The pipeline controller is governed by an FSM with three operational states:

- **normal**: The core asserts `instr_req` $\leq$ '1' toward its dedicated scheduler FIFO. When `instr_valid` is sampled high, the 64-bit instruction word is latched. The opcode decoder evaluates bits [3:0]:
 - If NOP, parameters are cleared and the core remains in **normal**.
 - If SETCOLOR, the 15-bit color field is stored in the internal color register `color_int` and the state remains **normal**.
 - If a drawing primitive or SWAP_BUFFERS is detected, the corresponding accelerator enable line in `acc_enable_vec`[5:0] is pulsed, the busy mask `acc_busy_vec` is set, and the FSM transitions to **drawing**.
- **drawing**: In this state, `instr_req` is held low to stall further instruction fetching. The core remains in **drawing** until the execution unit signals completion via `finish_exec` = '1', at which point the state transitions back to **normal**.
- **halt**: An asynchronous stall state entered whenever the external signal `core_halt` is asserted.

6.2.2 Parameter Registration

During decode, the packed bitfields of the instruction word are extracted and registered into dedicated 9-bit coordinate buses: (X_1, Y_1) , (X_2, Y_2) , (X_3, Y_3) , depth Z , and the active 15-bit RGB color.

6.3 Execution Unit and Tile Boundary Protection (spEXEC.vhd)

The spEXEC module interfaces the control signals from spPIPE to the physical rasterization engines.

6.3.1 Candidate Pixel Multiplexing and Completion Aggregation

When a drawing primitive is active, its corresponding hardware accelerator outputs a continuous stream of candidate pixels (X, Y, Color) qualified by an individual `pixel_valid` flag. The multiplexer process `pixel_out_proc` routes the active accelerator's outputs to the internal bus based on `acc_busy_vec`.

Simultaneously, the completion signals from all six accelerators and the swap controller are logically OR-ed to generate `finish_exec`:

$$\text{finish_exec} = \bigvee_{k=0}^5 \text{acc_finish_vec}(k) \vee \text{finish_swap} \quad (6.1)$$

This single-cycle pulse informs spPIPE that the primitive has been completely rasterized, allowing the core to fetch the next instruction from its FIFO.

6.3.2 Hardware Tile Clipping Unit

While the upstream scheduler restricts instruction dispatch to overlapping tiles, primitives that cross multiple tiles will naturally produce pixels outside the vertical bounds of any single tile.

To prevent out-of-bounds pixels from contaminating adjacent tile memory blocks or causing address wrap-around, spEXEC implements a dedicated, zero-latency **Tile Clipping Process** (`pixel_clip_proc`):

```

1 pixel_clip_proc : process(pixel_x_int, pixel_y_int, pixel_color_int, z_i,
2   pixel_valid_int, tile_index)
3   variable x_coord, y_coord : integer;
4   variable y_min, y_max    : integer;
5 begin
6   x_coord := to_integer(unsigned(pixel_x_int));
7   y_coord := to_integer(unsigned(pixel_y_int));
8
9   y_min := to_integer(unsigned(tile_index)) * 24;
10  y_max := y_min + 23;
11
12  pixel_x_o    <= pixel_x_int;
13  pixel_y_o    <= pixel_y_int;
14  pixel_color_o <= pixel_color_int;
15  pixel_z_o    <= z_i;
16
17  if pixel_valid_int = '1' then
18    if (x_coord >= 0 and x_coord < 320) and (y_coord >= y_min and y_coord <=
19      y_max) then
20      pixel_valid_o <= '1';
21    else
22      pixel_valid_o <= '0'; -- Zero-overhead hardware suppression
23    end if;
24  else
25    pixel_valid_o <= '0';
26  end if;
27 end process pixel_clip_proc;

```

Listing 6.1: Tile Boundary Enforcement in spEXEC.vhd.

If a candidate pixel's Y coordinate satisfies $Y \in [i \times 24, i \times 24 + 23]$, it is written into VRAM. If it falls outside this 24-line boundary, `pixel_valid_o` is driven to '0', silently discarding the pixel without interrupting the accelerator pipeline.

Hardware Primitive Accelerators

7.1 Overview of Dedicated Acceleration Engines

The core computational strength of spGPU resides in its suite of specialized, integer-arithmetic hardware accelerators instantiated within each spCORE. Rather than relying on a general-purpose ALU to execute iterative software rasterization loops, these units are synthesized directly as dedicated finite state machines and data-path pipelines.

Figure 7.1 presents the internal mathematical structures and operational data paths of the key acceleration engines.

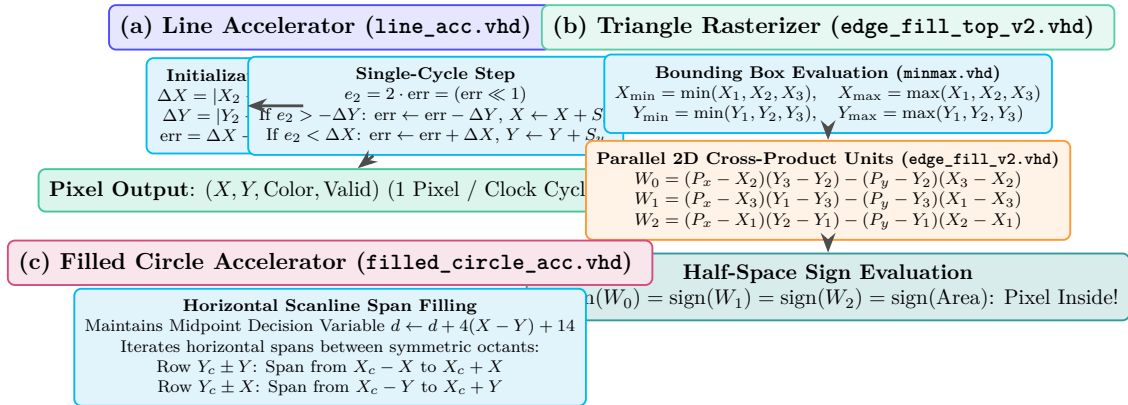


Figure 7.1: Mathematical Data Paths of the Hardware Accelerators: (a) Bresenham Line Unit, (b) Triangle Edge Equations Rasterizer, and (c) Filled Circle Scanline Generator.

7.2 Direct Pixel Writer (DRAWPIXEL)

The DRAWPIXEL operation is executed as a single-cycle direct register latch. When acc_enable_vec(0) is asserted, the execution unit registers X_1 and Y_1 directly to the output buses, asserts pixel_valid_wire(0) $\leq '1'$, and pulses acc_finish_vec(0) $\leq '1'$ in the very next clock cycle. This yields a single-cycle write latency.

7.3 Bresenham Line Accelerator (`line_acc.vhd`)

The line drawing engine implements an incremental, integer-only version of **Bresenham's Line Algorithm**.

7.3.1 Mathematical Formulation and Data Path

Given two endpoints (X_1, Y_1) and (X_2, Y_2) , the hardware computes:

$$\Delta X = |X_2 - X_1|, \quad \Delta Y = |Y_2 - Y_1| \quad (7.1)$$

$$S_x = \begin{cases} +1 & \text{if } X_1 < X_2 \\ -1 & \text{otherwise} \end{cases}, \quad S_y = \begin{cases} +1 & \text{if } Y_1 < Y_2 \\ -1 & \text{otherwise} \end{cases} \quad (7.2)$$

The decision error variable is initialized as $\text{err}_0 = \Delta X - \Delta Y$.

During the active **DRAW** state, the unit evaluates $e_2 = 2 \cdot \text{err}$ (computed via a zero-cost 1-bit left shift) and updates the coordinates and error accumulator in a single clock cycle:

$$\text{If } e_2 > -\Delta Y : \quad \text{err} \leftarrow \text{err} - \Delta Y, \quad X \leftarrow X + S_x \quad (7.3)$$

$$\text{If } e_2 < \Delta X : \quad \text{err} \leftarrow \text{err} + \Delta X, \quad Y \leftarrow Y + S_y \quad (7.4)$$

The accelerator outputs exactly **one pixel per clock cycle** along the line path until $(X, Y) = (X_2, Y_2)$, at which point it transitions to **DONE** and asserts `finish <= '1'`.

7.4 Midpoint Circle Accelerator (`circle_acc.vhd`)

The hollow circle engine implements the **Midpoint Circle Algorithm**, exploiting the 8-fold rotational and reflective symmetry of the circle.

7.4.1 Eight-Octant Traversal

Given center (X_c, Y_c) and radius R , the decision variable is initialized to $d = 3 - 2R$. For each radial step along the first octant, the FSM traverses states **DRAW1** through **DRAW8**, emitting the eight symmetric points across eight consecutive clock cycles:

$$(X_c + X, Y_c + Y), \quad (X_c - X, Y_c + Y), \quad (X_c + X, Y_c - Y), \quad (X_c - X, Y_c - Y) \quad (7.5)$$

$$(X_c + Y, Y_c + X), \quad (X_c - Y, Y_c + X), \quad (X_c + Y, Y_c - X), \quad (X_c - Y, Y_c - X) \quad (7.6)$$

The decision parameter d is subsequently updated using addition and shift operations:

$$d_{k+1} = \begin{cases} d_k + 4(X - Y) + 10, & Y \leftarrow Y - 1 & \text{if } d_k > 0 \\ d_k + 4X + 6 & & \text{if } d_k \leq 0 \end{cases} \quad (7.7)$$

The cycle repeats until $X > Y$.

7.5 Filled Circle Accelerator (`filled_circle_acc.vhd`)

To render solid circular discs, **F_CIRCLE_ACC** extends the midpoint circle algorithm into a **Horizontal Span Rasterizer**.

Instead of plotting isolated boundary pixels, the engine utilizes horizontal scanlines that sweep continuously between opposing symmetric octants:

- **Vertical Octant Pair:** Sweeps horizontal index i from $(X_c - X)$ to $(X_c + X)$ across the two rows $Y_c + Y$ and $Y_c - Y$.
- **Horizontal Octant Pair:** Sweeps horizontal index j from $(X_c - Y)$ to $(X_c + Y)$ across the two rows $Y_c + X$ and $Y_c - X$.

This dual-span interpolation guarantees complete geometric coverage with zero interior voids, emitting one filled pixel per clock cycle across each horizontal span.

7.6 Filled Triangle Rasterizer (`edge_fill_top_v2.vhd`)

The triangle fill accelerator utilizes the **Half-Space Edge Equations Method** (equivalent to 2D cross-products or barycentric orientation testing).

7.6.1 Bounding Box Determination (`minmax.vhd`)

Before rasterizing, the combinational module `minmax.vhd` evaluates the 2D bounding box of the three triangle vertices:

$$X_{\min} = \min(X_1, X_2, X_3), \quad X_{\max} = \max(X_1, X_2, X_3) \quad (7.8)$$

$$Y_{\min} = \min(Y_1, Y_2, Y_3), \quad Y_{\max} = \max(Y_1, Y_2, Y_3) \quad (7.9)$$

Rasterization scanning is strictly confined within this bounding rectangle $[X_{\min}, X_{\max}] \times [Y_{\min}, Y_{\max}]$, avoiding unnecessary computation across the rest of the canvas.

7.6.2 Parallel Edge Function Evaluation (`edge_fill_v2.vhd`)

For any point $P = (X, Y)$, the oriented edge function $E(P)$ of the directed line segment $(X_a, Y_a) \rightarrow (X_b, Y_b)$ is defined as:

$$E(P) = (P_x - X_a)(Y_b - Y_a) - (P_y - Y_a)(X_b - X_a) \quad (7.10)$$

Three parallel instances of `edge_fill_v2` compute the edge functions W_0, W_1, W_2 for the three edges of the triangle simultaneously:

$$W_0 = (X - X_2)(Y_3 - Y_2) - (Y - Y_2)(X_3 - X_2) \quad (7.11)$$

$$W_1 = (X - X_3)(Y_1 - Y_3) - (Y - Y_3)(X_1 - X_3) \quad (7.12)$$

$$W_2 = (X - X_1)(Y_2 - Y_1) - (Y - Y_1)(X_2 - X_1) \quad (7.13)$$

A fourth instance computes the reference oriented area of the full triangle:

$$\text{Area} = (X_3 - X_1)(Y_2 - Y_1) - (Y_3 - Y_1)(X_2 - X_1) \quad (7.14)$$

7.6.3 Half-Space Inclusion Test

A candidate point $P(X, Y)$ lies strictly inside the triangle if and only if all three edge functions have the same sign as the total oriented area:

$$P \in \triangle \iff (\text{sign}(W_0) = \text{sign}(\text{Area})) \wedge (\text{sign}(W_1) = \text{sign}(\text{Area})) \wedge (\text{sign}(W_2) = \text{sign}(\text{Area})) \quad (7.15)$$

When this condition evaluates true, `pixel_valid <= '1'` is asserted and the pixel is written to VRAM. The rasterizer then advances $X \leftarrow X + 1$; upon reaching $X_{\max}$, it resets $X \leftarrow X_{\min}$ and increments $Y \leftarrow Y + 1$ until the entire bounding box has been processed.

Chapter 8

Tile Framebuffer, Double Buffering, and Hardware Z-Buffer

8.1 Memory Architecture Overview

Each of the ten horizontal tiles is backed by an independent on-chip memory subsystem implemented in `tile.vhd`. The module integrates:

- A **Color VRAM** organized as a True Dual-Port Block RAM supporting simultaneous write operations at 100.0 MHz and display scan read operations at 25.0 MHz.
- A **Ping-Pong Double Buffering** mechanism that isolates the active rendering target from the scanning display.
- An integrated **Hardware Z-Buffer (Depth Buffer)** providing single-cycle depth testing per pixel.
- A **Hardware Auto-Clear Engine** that resets the back buffer to background white and resets depth to maximum distance with zero CPU cycle consumption.
- A **Multiplier-Free Address Generator** utilizing arithmetic bit shifts in place of DSP multipliers.

Figure 8.1 illustrates the complete data-path and control architecture of `tile.vhd`.

8.2 True Dual-Port BRAM Organization and Double Buffering

Each tile canvas has a resolution of $320 \times 24 = 7,680$ pixels. To support double buffering, the color VRAM stores two complete frames:

$$\text{VRAM Size} = 2 \times (320 \times 24) = 15,360 \text{ words} \times 15 \text{ bits} \quad (8.1)$$

The ping-pong register `current_fb` controls the base address offsets for read and write ports:

- When `current_fb = '0'`: The compute core writes into the **Back Buffer** (starting at address offset 7,680), while the video display scans from the **Front Buffer** (starting at address offset 0).

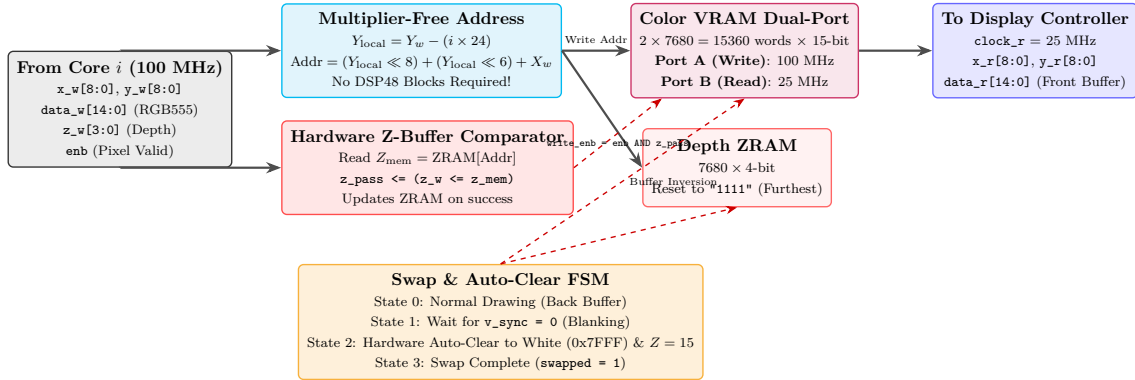


Figure 8.1: Microarchitecture of the Tile Framebuffer and Integrated Z-Buffer (`tile.vhd`).

- When `current_fb = '1'`: The compute core writes into the **Back Buffer** (offset 0), while the video display scans from the **Front Buffer** (offset 7,680).

8.3 Swap Protocol and Hardware Background Auto-Clear

The buffer swapping sequence is coordinated through a 4-state FSM in each tile:

1. **State 0 (Drawing)**: Normal rendering into the back buffer. The tile monitors `fb_swap`.
2. **State 1 (Wait for VSYNC)**: When all 10 cores have processed the `SWAP_BUFFERS` instruction, the global signal `fb_swap_general` goes high. The FSM waits for the vertical blanking interval of the display (`v_sync_sync = '0'`). Once detected, `current_fb` is toggled:

$$\text{current_fb} \leftarrow \neg \text{current_fb} \quad (8.2)$$

Because the pointer inversion occurs strictly during vertical blanking, screen tearing is completely eliminated.

3. **State 2 (Hardware Auto-Clear)**: During this state, an internal counter `clear_cnt` increments from 0 to 7,679, sweeping the newly assigned back buffer. At each cycle, the hardware writes white (15-bit "111111111111111") into VRAM and maximum distance ("1111") into ZRAM.
4. **State 3 (Swap Complete)**: Asserts `swapped <= '1'` to inform `spEXEC` that the new buffer is clean and ready for rendering.

8.4 Integrated 4-Bit Hardware Z-Buffer (Depth Test)

To enable proper 2.5D and 3D visual layering without requiring the host CPU to sort polygons from back-to-front (Painter's Algorithm), each tile memory module embeds a dedicated 4-bit depth memory (ZRAM) of 7,680 entries.

8.4.1 Single-Cycle Depth Comparator

On every clock cycle where a candidate pixel is asserted (`enb = '1'`), the hardware performs an instantaneous read-compare-write cycle:

$$Z_{\text{current}} = \text{ZRAM}[\text{address_w}] \quad (8.3)$$

$$z_pass = \begin{cases} '1' & \text{if } Z_{in} \leq Z_{current} \\ '0' & \text{otherwise} \end{cases} \quad (8.4)$$

$$write_enb = enb \wedge hit_w \wedge z_pass \quad (8.5)$$

If the candidate pixel's depth Z_{in} is less than or equal to the current depth stored in ZRAM (indicating that the new primitive is closer to or coplanar with the viewer), `write_enb` is asserted: the pixel color is committed to VRAM, and ZRAM is updated with Z_{in} . Otherwise, the write is suppressed, effectively culling obscured fragments at zero performance penalty.

8.5 Multiplier-Free Address Generation

In standard architectures, mapping 2D coordinates (X, Y) to a linear 1D memory array requires integer multiplication:

$$Addr = Y_{local} \times 320 + X \quad (8.6)$$

Implementing ten such multipliers in parallel would consume 10 to 20 DSP48 slices solely for address calculation.

spGPU circumvents this by decomposing 320 into powers of two ($320 = 256 + 64 = 2^8 + 2^6$):

$$Addr = (Y_{local} \ll 8) + (Y_{local} \ll 6) + X \quad (8.7)$$

Similarly, the tile vertical offset $Y_{offset} = tile_index \times 24$ is decomposed as $24 = 16 + 8 = 2^4 + 2^3$:

$$Y_{offset} = (tile_index \ll 4) + (tile_index \ll 3) \quad (8.8)$$

As shown in Listing 8.1, this enables the synthesizer to map address generation entirely to barrel shifters and carry-chain adders, conserving all FPGA DSP48 blocks for geometry and filtering.

```

1  -- Offset for the tile: tile_index * 24 = (tile_index * 16) + (tile_index * 8)
2  y_offset <= shift_left(resize(unsigned(tile_index), N_PIXEL), 4) +
3              shift_left(resize(unsigned(tile_index), N_PIXEL), 3);
4
5  -- Local address calculation: Y_local * 320 + X = (Y_local * 256) + (Y_local *
6      64) + X
7  address_w <= std_logic_vector(
8      shift_left(resize(y_w_local, 18), 8) +
9      shift_left(resize(y_w_local, 18), 6) +
10     resize(unsigned(x_w), 18)
11 );

```

Listing 8.1: Shift-and-Add Memory Address Generation in `tile.vhd`.

Chapter 9

Display Controller and Physical HDMI Transmitter

9.1 Video Subsystem Overview

To eliminate the need for external digital-to-analog converters (such as VGA DAC chips) or third-party HDMI bridge controllers, spGPU incorporates an entirely native, hardware-synthesized **HDMI Transmitter and Display Pipeline**.

The subsystem interfaces directly with the physical differential pins of the HDMI Type-A connector on the PYNQ-Z1 board. Figure 9.1 traces the signal path from the VGA timing generator to the differential output buffers.

9.2 VGA Timing Generation (`counter_pixel.vhd`)

The master video timing is produced by `counter_pixel.vhd` operating at a pixel clock frequency of 25.0 MHz (standard 640×480 @ 60 Hz VGA progressive timing):

- **Horizontal Parameters:** 640 active display pixels, Front Porch = 16, HSYNC pulse = 96, Back Porch = 48, giving a total line period of 800 pixel clock cycles ($32.0 \mu\text{s}$, 31.25 kHz line rate).
- **Vertical Parameters:** 480 active display scanlines, Front Porch = 10, VSYNC pulse = 2, Back Porch = 33, yielding a total frame period of 525 lines (16.8 ms, ≈ 59.52 Hz refresh rate).

9.3 Hardware $2\times$ Nearest-Neighbor Upscaling

The internal rendering resolution of spGPU is 320×240 , whereas the HDMI transmitter requires a 640×480 raster.

Rather than consuming memory bandwidth or DSP resources with bilinear filtering buffers, spGPU executes a zero-latency, zero-cost **$2\times$ Nearest-Neighbor Upscaling** directly in hardware by truncating the least significant bit (LSB) of the 10-bit VGA counter coordinates:

$$H_{\text{gpu}}[8 : 0] = H_{\text{vga}}[9 : 1] \implies H_{\text{gpu}} = \lfloor H_{\text{vga}}/2 \rfloor \quad (9.1)$$

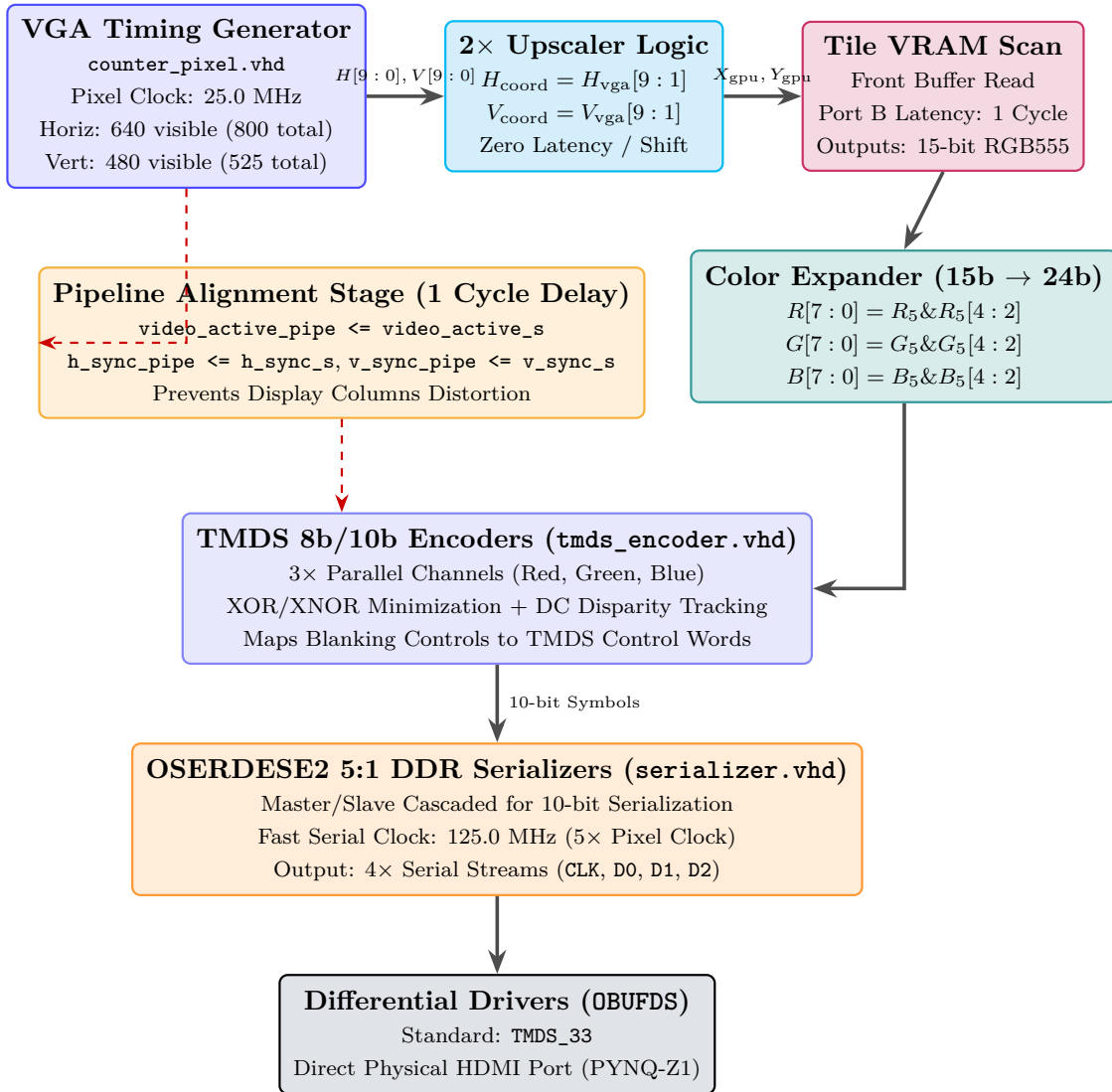


Figure 9.1: Complete HDMI Display and Physical Transmission Pipeline.

$$V_{\text{gpu}}[8:0] = V_{\text{vga}}[9:1] \implies V_{\text{gpu}} = \lfloor V_{\text{vga}}/2 \rfloor \quad (9.2)$$

Each internal pixel rendered into VRAM is thus held for two consecutive pixel clocks horizontally and two consecutive scanlines vertically, creating a sharp, pixel-perfect 2×2 pixel block on the high-definition monitor.

9.4 BRAM Pipeline Latency Alignment

Synchronous Xilinx Block RAM primitives introduce an intrinsic 1-clock-cycle read latency between address presentation and valid data output. If the video synchronization pulses (`h_sync`, `v_sync`) and the active video flag (`video_active`) were transmitted concurrently with address generation, the displayed image would be horizontally shifted by one pixel and the rightmost column would be truncated.

The module `vga_struct.vhd` resolves this by routing the timing flags through dedicated 1-cycle pipeline registers:

```

1 align_proc : process(pixelclock, rst)
2 begin
3     if rst = '1' then
4         video_active_pipe <= '0';
5         h_sync_pipe      <= '1';
6         v_sync_pipe      <= '1';
7     elsif rising_edge(pixelclock) then
8         video_active_pipe <= video_active_s;
9         h_sync_pipe      <= h_sync_s;
10        v_sync_pipe      <= v_sync_s;
11    end if;
12 end process;
    
```

Listing 9.1: Pipeline Latency Alignment in `vga_struct.vhd`.

This guarantees that pixel color data leaving the BRAM arrives at the TMDS encoder in exact temporal alignment with its corresponding active video framing.

9.5 Color Expansion (RGB555 to RGB888)

The 15-bit color stored in VRAM allocates 5 bits per primary color. To drive standard 24-bit TrueColor HDMI sinks without darkening the dynamic range, the 5-bit color components are expanded to 8 bits by replicating the 3 most significant bits into the 3 least significant bit positions:

$$R_8[7:0] = R_5[4:0] \parallel R_5[4:2] \quad (9.3)$$

$$G_8[7:0] = G_5[4:0] \parallel G_5[4:2] \quad (9.4)$$

$$B_8[7:0] = B_5[4:0] \parallel B_5[4:2] \quad (9.5)$$

This ensures that maximum 5-bit value ("11111") maps accurately to maximum 8-bit value ("11111111" = 255), maintaining full dynamic contrast and color saturation.

9.6 TMDS 8b/10b Encoders (`tmds_encoder.vhd`)

Transition-Minimized Differential Signaling (TMDS) encodes 8-bit color data into 10-bit symbols to minimize high-frequency electromagnetic interference (EMI) and maintain DC balance over the physical cable:

1. **Transition Minimization:** Evaluates the number of '1' bits in the input byte. Depending on the density, it generates an 8-bit sequence using either cumulative XOR or cumulative XNOR operations, asserting the 9th bit (`q_m[8]`) to denote the chosen scheme.
2. **DC Balance & Disparity Tracking:** An internal signed 4-bit accumulator (`disparity`) monitors the running discrepancy between transmitted '1's and '0's. If the disparity exceeds zero, the 10th bit is inverted to restore the DC baseline of the transmission line.
3. **Blanking Control Word Mapping:** During the horizontal and vertical blanking intervals (`video_active = '0'`), the encoder maps the 2-bit control inputs (`h_sync`, `v_sync`) to one of four unique 10-bit synchronization sequences ("1101010100", "0010101011", etc.) exhibiting high transition rates for phase-locked loop (PLL) clock recovery at the sink.

9.7 Physical Serialization and Differential I/O (`serializer.vhd`)

To transmit 10-bit parallel TMDS symbols over a single-conductor pair per channel at 60 Hz, the bit rate per lane is:

$$\text{Bit Rate} = 25.0 \text{ MHz} \times 10 = 250.0 \text{ Mbps/lane} \quad (9.6)$$

In Xilinx 7-series FPGAs, this is realized by cascading two **OSERDESE2** hardware primitives in Master/Slave configuration:

- **Clocking:** Operating in Double Data Rate (DDR) mode with a 5:1 serialization ratio, the blocks are clocked by the fast serial clock `serialclock = 125.0 MHz` ($5 \times 25.0 \text{ MHz}$) and divided parallel clock `pixelclock = 25.0 MHz`.
- **Cascading:** The Master OSERDESE2 serializes the lower 8 bits, while the Slave OSERDESE2 supplies the upper 2 bits via internal shift lines (`SHIFTIN1`, `SHIFTIN2`).
- **Differential Drivers:** The serialized output bitstreams for the clock channel and the three data channels (Red, Green, Blue) drive four Xilinx **OBUFDS** differential output buffer primitives configured with the `TMDS_33` I/O standard, providing the correct current and impedance matching for direct connection to any standard HDMI monitor.

Chapter 10

Performance Analyzer and Dual-Interrupt Architecture

10.1 Hardware Telemetry Subsystem (`spANALYZER.vhd`)

To obtain real-time, non-intrusive measurements of GPU frame rate without skewing execution times with software timestamp queries, spGPU incorporates a dedicated hardware performance telemetry core: **spANALYZER**.

The module operates within the 100.0 MHz core clock domain and maintains two synchronous counters:

1. **High-Precision 1-Second Timebase:** A 27-bit counter (`sync_vector`) counts clock pulses continuously up to a terminal count of $TC = 99,999,999$. When the terminal count is reached, a 1-clock-cycle pulse `one_sec_elapsed` is asserted exactly once every 1.000000 s.
2. **Frame Swap Accumulator:** A 10-bit counter (`swap_vector`) increments every time the GPU completes a full double-buffer swap (`swapped = '1'`).

When `one_sec_elapsed` fires, the accumulated swap count is latched into the output register `fps_int_reg`, the swap accumulator is reset, and an interrupt pulse is emitted on `int_pin`. The 10-bit `fps` value is routed through an AXI GPIO peripheral to allow instantaneous memory-mapped readout by the CPU.

10.2 Dual-Interrupt Hardware Architecture

A central innovation in the system's software-hardware co-design is the **Dual-Interrupt Synchronization Architecture**. Two distinct hardware events generate rising-edge interrupts to the ARM Generic Interrupt Controller (GIC), as mapped in Table 10.1.

10.3 Clock Domain Crossing for VSYNC Pulse Generation

The VGA vertical synchronization signal `v_sync_int` originates in the 25.0 MHz pixel clock domain. Connecting this asynchronous signal directly to the 100.0 MHz GIC interrupt line

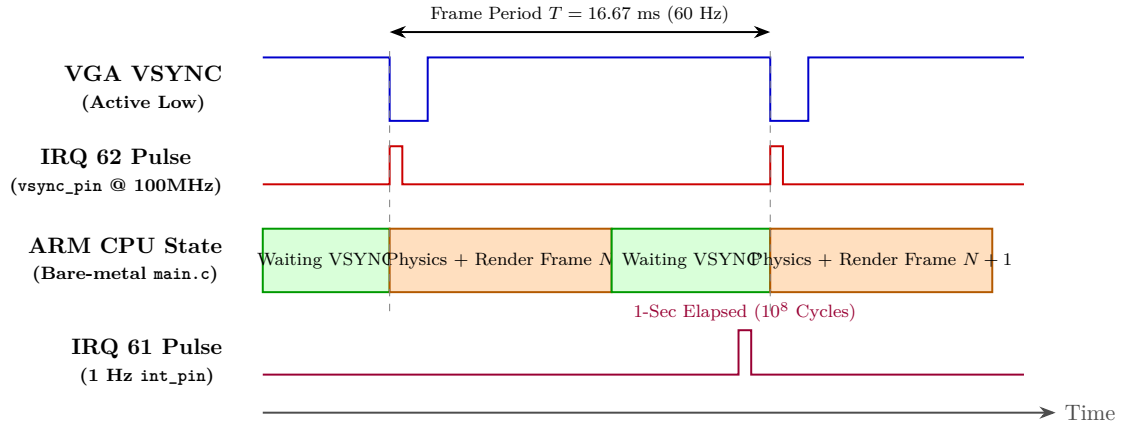


Figure 10.1: Dual-Interrupt Hardware/Software Synchronization and Frame Timing Diagram.

Table 10.1: Hardware Interrupt Mapping to ARM Cortex-A9 GIC

GIC IRQ ID	Hardware Pin	Trigger Mode	Handler Function / System Behavior
61	spANALYZER / int_pin	Rising Edge (0x03)	Core0nIRQ_Handler: Fires once per second (1 Hz). Reads the hardware FPS register from AXI GPIO and sets <code>g_fps_ready = 1</code> for console logging.
62	spgpu / vsync_pin	Rising Edge (0x03)	VSynch_Handler: Fires at display refresh rate (60 Hz). Synchronized to the falling edge of VGA VSYNC, setting <code>g_vsync_occurred = 1</code> to unlock the physics loop.

would introduce metastability hazards and multiple trigger events.

In `sp-gpu.vhd`, a double flip-flop synchronizer combined with an edge-detection filter safely transfers the signal into the 100.0 MHz core domain:

```

1 process(clk, rst)
2 begin
3     if rst = '1' then
4         vsync_meta    <= '1';
5         vsync_sync    <= '1';
6         vsync_sync_d  <= '1';
7         vsync_pulse   <= '0';
8     elsif rising_edge(clk) then
9         vsync_meta    <= v_sync_int;
10        vsync_sync    <= vsync_meta;
11        vsync_sync_d  <= vsync_sync;
12
13        -- Generate a single 10-ns pulse on the active-low falling edge of VSYNC
14        if vsync_sync_d = '1' and vsync_sync = '0' then
15            vsync_pulse <= '1';
16        else
17            vsync_pulse <= '0';
18        end if;
19    end if;
20 end process;
21 vsync_pin <= vsync_pulse;

```

Listing 10.1: Clock Domain Crossing and VSYNC Pulse Generator in `sp-gpu.vhd`.

This circuit emits an exact, single-cycle 10-nanosecond pulse at the beginning of every vertical blanking interval, ensuring clean edge-triggering within the ARM interrupt controller.

10.4 Frame Pacing and Zero-Tearing Lock

As visualized in Figure 10.1, this dual-interrupt mechanism solves the problem of frame pacing:

1. At the start of each frame, the ARM processor invokes the blocking function `WaitForVsync()`:

```

1 void WaitForVsync(void) {
2     while (!g_vsync_occurred) {
3         // Suspends execution until hardware IRQ 62 triggers
4     }
5     g_vsync_occurred = 0;
6 }
7

```

Listing 10.2: Hardware VSYNC Synchronization Function in `main.c`.

2. When IRQ 62 arrives, the CPU awakens, updates the physical positions and velocities of the scene objects with an exact time step of $\Delta t = 1/60\text{ s} \approx 16.67\text{ ms}$, issues the rendering commands via DMA, and calls `SwapBuffers()`.
3. Because the frame time budget is strictly bounded by the 60 Hz VSYNC pulse, the CPU can never outrun the display or flood the GPU command FIFOs. Rendering occurs exclusively into the back buffer, and buffer swap occurs strictly within the blanking window, yielding rock-solid 60 FPS display with zero image tearing.

Chapter 11

Bare-Metal Software Stack and Host Driver

11.1 Software Architecture Overview

The software stack operating on the ARM Cortex-A9 is structured in two primary layers:

1. **spGPU Driver Library (sp_lib.c and sp_lib.h)**: Provides low-level hardware abstraction, DMA register management, coordinate validation, and zero-copy 64-bit instruction bit-packing.
2. **Physics Application Demo (main.c)**: Implements an interactive multi-body physics simulation featuring 10 bouncing spheres with continuous collision response, boundary clamping, and real-time hardware telemetry display.

11.2 Low-Level AXI DMA Driver

The driver interfaces with the Xilinx AXI DMA engine through direct register read/write macros (`Xil_In32`, `Xil_Out32`):

- **Initialization (InitDMA)**: Enables the MM2S DMA channel by setting the run/stop bit (`RS = 1`) in the control register:

```
1  void InitDMA(void) {  
2      unsigned int tmpVal = Xil_In32(XPAR_AXIDMA_0_BASEADDR + 0x0);  
3      tmpVal = tmpVal | 0x1001; // Enable DMA run and interrupts  
4      Xil_Out32(XPAR_AXIDMA_0_BASEADDR + 0x0, tmpVal);  
5  }  
6
```

Listing 11.1: DMA Controller Initialization in `sp_lib.c`.

- **Asynchronous Transfer Initiation (StartDMATransfer)**: Configures the source address register (`0x18`) with the pointer to the instruction structure and writes the byte length into the transfer length register (`0x28`). The function polls the status register (`0x04`) until bit 12 (`IOC_Irq`) confirms transfer completion.

11.3 Zero-Copy Bit-Packed Instruction Mapping

To construct 64-bit spISA instructions efficiently without manual bitmasking and shifting on every call, the driver utilizes C bitfield unions declared with the GCC attribute `__attribute__((packed))`.

As shown in Listing 11.2, the compiler maps the C structure fields directly into the exact 64-bit memory layout expected by the FPGA hardware.

```

1 typedef union {
2     struct __attribute__((packed)) {
3         uint64_t opcode : 4; // Bits 0-3: Opcode (0b0010 = DrawLine)
4         uint64_t z      : 4; // Bits 4-7: Depth Z
5         uint64_t x1     : 9; // Bits 8-16: X1 coordinate
6         uint64_t y1     : 9; // Bits 17-25: Y1 coordinate
7         uint64_t x2     : 9; // Bits 26-34: X2 coordinate
8         uint64_t y2     : 9; // Bits 35-43: Y2 coordinate
9         uint64_t padding : 20; // Bits 44-63: Unused padding
10    };
11    uint64_t instr; // 64-bit assembled instruction word
12 } DrawLine_t;
13
14 typedef union {
15     struct __attribute__((packed)) {
16         uint64_t opcode : 4; // Bits 0-3: Opcode (0b0110 = DrawCircleF)
17         uint64_t z      : 4; // Bits 4-7: Depth Z
18         uint64_t xc     : 9; // Bits 8-16: Xc center coordinate
19         uint64_t yc     : 9; // Bits 17-25: Yc center coordinate
20         uint64_t r      : 9; // Bits 26-34: Radius R
21         uint64_t padding : 29; // Bits 35-63: Unused padding
22    };
23    uint64_t instr;
24 } DrawCircle_t;

```

Listing 11.2: Packed C Unions for spISA Encoding in `sp_lib.h`.

11.4 High-Level Drawing API

The driver encapsulates instruction emission within high-level function calls. Each drawing function performs boundary checking against the display resolution (320×240) and issues a paired `SetColor` instruction before transmitting the primitive:

```

1 int DrawCircleF(uint64_t xc, uint64_t yc, uint64_t r, unsigned int color,
2   unsigned char z) {
3     DrawCircle_t instr = {0};
4     if (xc >= VIDEO_X || yc >= VIDEO_Y) {
5         printf("Coordinates exceed screen dimensions\n");
6         return 1;
7     }
8     instr.opcode = 6; // OP = 0b0110
9     instr.xc = xc;
10    instr.yc = yc;
11    instr.r = r;
12    instr.z = z;
13
14    if (SetColor(color)) return 1;
15    StartDMATransfer(&instr, sizeof(DrawCircle_t));
16    return 0;
17 }

```

Listing 11.3: Filled Circle Drawing Implementation in `sp_lib.c`.

11.5 Real-Time Physics Demonstration Application (`main.c`)

To benchmark the system under heavy graphical workloads, the demonstration application simulates 10 elastic circular bodies interacting under Newtonian mechanics.

11.5.1 Physical Model and Collision Resolution

Each sphere i possesses a state vector $\mathbf{S}_i = [x, y, v_x, v_y, m, r, \text{col}]^T$.

When two spheres a and b collide ($\|\mathbf{p}_b - \mathbf{p}_a\| \leq r_a + r_b$), the overlap is corrected along the collision normal $\mathbf{n} = (\mathbf{p}_b - \mathbf{p}_a) / \|\mathbf{p}_b - \mathbf{p}_a\|$, and the elastic impulse scalar j is applied:

$$j = \frac{-(1+e)(\mathbf{v}_{\text{rel}} \cdot \mathbf{n})}{\frac{1}{m_a} + \frac{1}{m_b}}, \quad e = 1.0 \text{ (Perfect Elasticity)} \quad (11.1)$$

$$\mathbf{v}_a \leftarrow \mathbf{v}_a - \frac{j}{m_a} \mathbf{n}, \quad \mathbf{v}_b \leftarrow \mathbf{v}_b + \frac{j}{m_b} \mathbf{n} \quad (11.2)$$

Boundary collisions against the four walls ($X \in [0, 319], Y \in [0, 239]$) are resolved with positional clamping to guarantee that circles never drift off-screen:

$$\text{If } x - r < 0 \implies x \leftarrow r, \quad v_x \leftarrow -v_x \quad (11.3)$$

$$\text{If } x + r \geq 320 \implies x \leftarrow 319 - r, \quad v_x \leftarrow -v_x \quad (11.4)$$

11.5.2 Execution Loop and Console Output

The application loop synchronizes to the 60 Hz VSYNC interrupt. Once per second, when IRQ 61 sets `g_fps_ready`, the system prints the measured performance metrics over the UART serial interface (115200 baud):

```
FPS (HW): 60 | Frames (VSYNC): 60
FPS (HW): 60 | Frames (VSYNC): 60
FPS (HW): 60 | Frames (VSYNC): 60
```

This confirms that the GPU comfortably rasterizes the ten overlapping filled circles, performs all depth tests, and completes the swap well within the 16.67 ms frame deadline.

Chapter 12

Simulation, Verification, and Tooling

12.1 Verification Strategy

Designing a complex multicore graphics processor requires multi-level verification to ensure mathematical fidelity, correct pipeline sequencing, and absence of visual artifacts. The verification framework for spGPU spans three complementary domains:

1. **C/C++ Algorithmic Golden Models:** Functional verification of integer rasterization math.
2. **VHDL RTL Simulation:** Cycle-accurate logic verification and waveform analysis in ModelSim / QuestaSim.
3. **Visual Testbench Co-Simulation:** Desktop software visualization using the *Raylib* engine to render simulation output files before physical FPGA synthesis.

12.2 Software Simulation Models (`software_sim/`)

Before implementing hardware descriptions in VHDL, software prototypes were built in C and C++ to validate the mathematical formulations:

- **`bresenhamLine.c`:** Validated the integer decision variable accumulator and step directions for all eight slope octants.
- **`bresenhamCircle.c` and `fill_circle.c`:** Verified octant reflection and scanline span continuity to ensure no missing pixels along horizontal seams.
- **`edge_fill.c` and `bf_triangle.cpp`:** Validated the 2D cross-product edge equations, evaluating orientation invariance for clockwise and counter-clockwise vertex definitions.
- **`nn_upscaling.cpp`:** An OpenCV-based model testing the $2\times$ nearest-neighbor scaling algorithm on sample photographic and synthetic bitmap inputs, confirming that coordinate downshifting preserves sharpness without visual distortion.

12.3 Interactive Visual Simulator (`simulator.c`)

A major challenge in RTL graphics verification is that inspecting thousands of raw pixel coordinates in logic analyzer waveforms (such as `vsim.wlf`) is tedious and cannot easily reveal subtle geometric flaws (e.g., rasterization gaps, edge aliasing, or clipping leaks).

To overcome this, the authors created `simulator.c`, a desktop visualization tool built using the open-source **Raylib** graphics library:

1. During VHDL simulation of the top-level testbench (`tb_spGPU.vhd`), a file logger process captures every asserted pixel:

$$(X, Y, \text{Color}) \implies \text{Logged to output_pixels.txt} \quad (12.1)$$

2. `simulator.c` parses the resulting text file, allocates an array of `PixelRecord` structures, and reconstructs the full visual frame inside an interactive 800×600 window on the developer's PC.
3. This visual co-simulation enabled immediate visual inspection of triangles, lines, and filled circles synthesized by the VHDL accelerators, drastically reducing debug cycles.

12.4 RTL Testbench Suite and ModelSim Automation

The repository includes a comprehensive hierarchy of VHDL testbenches in the `tb/` directory:

- **Unit Testbenches:**

- `tb_line_acc.vhd`: Stimulates the line accelerator with various slope combinations (steep, shallow, negative, zero length).
- `tb_circle_acc.vhd` and `tb_filled_circle_acc.vhd`: Verifies circle rasterization for various radii and off-center coordinate offsets.
- `tb_edge_fill.vhd` and `tb_edge_fill_top.vhd`: Stimulates the triangle fill unit with acute, obtuse, and degenerate triangles.

- **Integration Testbenches:**

- `tb_spCORE.vhd`: Verifies instruction fetch, pipeline sequencing, and tile clipping on a single core.
- `tb_spGPU.vhd`: Full-chip simulation with instruction streaming from `instr_input.txt` and pixel dumping to `output_pixels.txt`.

- **Automated Compilation Scripts:** The script `compile/compile.sh` automates library mapping (`vlib`, `vmap`), compiles all RTL files in strict dependency order with VHDL-2008 compatibility (`vcom -2008`), and launches QuestaSim/ModelSim with waveform profiling configurations (`waves.tcl`).

Chapter 13

Implementation Results and Design Discussion

13.1 FPGA Synthesis and Resource Utilization

The spGPU multicore architecture was synthesized and implemented targeting the **Xilinx Zynq-7000 SoC (XC7Z020-1CLG400C)** using AMD Vivado 2023.1.

Table 13.1 provides an architectural breakdown of the FPGA resource footprint across the primary functional blocks.

Table 13.1: Estimated FPGA Resource Utilization on Xilinx Zynq XC7Z020

Component / Module	LUTs	Flip-Flops (FF)	BRAM36E1 (Tiles)	DSP48E1
10× Compute Cores (spCORE)	≈ 14,500	≈ 6,800	0	0
10× Tile Framebuffers (tile.vhd)	≈ 3,200	≈ 1,800	64 (140 blocks)	0
Dynamic Scheduler (spScheduler)	≈ 1,600	≈ 850	0	0
Performance Telemetry (spANALYZER)	≈ 150	≈ 120	0	0
Display Controller & HDMI TX	≈ 850	≈ 650	0	0
AXI DMA & Stream Bridge	≈ 1,400	≈ 1,600	4	0
Total spGPU System	≈ 21,700	≈ 11,820	≈ 68	0
Available on XC7Z020	53,200	106,400	140	220
Utilization Percentage	≈ 40.8%	≈ 11.1%	≈ 48.6%	0.0%

13.2 Clock Domain Architecture and Timing Closure

The system operates across three distinct clock domains synthesized from the onboard 125.0 MHz reference oscillator via a mixed-mode clock manager (MMCM):

1. **GPU Core Clock (100.0 MHz, Period = 10.0 ns):** Drives the AXI4-Stream slave interface, the instruction scheduler, all 10 compute cores, and Port A (write) of the tile VRAMs. Timing closure is met with positive worst negative slack (WNS > +1.2 ns).

2. **Video Pixel Clock (25.0 MHz, Period = 40.0 ns):** Drives the VGA timing counters, the scanline output multiplexer, Port B (read) of the tile VRAMs, and the parallel stage of the TMDS encoders.
3. **HDMI Serial Clock (125.0 MHz, Period = 8.0 ns):** Provides the high-speed clock for the USERDESE2 5:1 DDR serializers, generating the 250 Mbps bitstreams for differential HDMI signaling.

Safe Clock Domain Crossing (CDC) is guaranteed through dedicated double-register synchronizers for status and swap handshake flags, eliminating metastability.

13.3 Analysis of Architectural Innovations and Trade-Offs

Several key design choices define the uniqueness of spGPU:

- **Zero DSP Usage for Memory Addressing:** By decomposing coordinate addresses into shift-and-add operations $((Y \ll 8) + (Y \ll 6) + X)$, the design completely avoids consuming DSP48 multipliers for framebuffer access, leaving all 220 DSP slices available for advanced audio processing or computer vision accelerators on the SoC.
- **On-Chip VRAM vs. External DRAM:** Storing the entire double-buffered framebuffer in on-chip Block RAM limits resolution to 320×240 , but completely eliminates external DDR memory latency, memory controller overhead, and bus contention.
- **Hardware Z-Buffering:** Integrating depth comparisons directly into the BRAM write port enables 2.5D visual layering at single-cycle latency, offloading complex depth sorting entirely from the CPU.

13.4 Potential Future Enhancements

Future iterations of spGPU could build upon this foundation to introduce:

1. **Subpixel Precision and Anti-Aliasing:** Extending the 9-bit coordinate registers to 12.4 fixed-point format to implement subpixel line positioning and coverage-based anti-aliasing (MSAA).
2. **Smooth Gouraud Shading:** Interpolating per-vertex color components across triangle primitives in `edge_fill_top_v2.vhd`.
3. **Hardware Texture Mapping Engine:** Adding a read-only texture cache BRAM to allow mapping compressed bitmap textures onto 2D polygons.
4. **Hardware Bit-Block Transfer (BitBLT):** Implementing an instruction for high-speed rectangular memory copy and alpha blending.

Chapter 14

Conclusion

The **spGPU** project represents a comprehensive and rigorous implementation of a modern, multicore, tile-based 2D/2.5D graphics processing unit synthesized on an FPGA System-on-Chip.

By designing every layer of the architecture from the ground up—spanning from the mathematical formulation of integer rasterization algorithms in software, through VHDL RTL modeling of parallel compute pipelines and true dual-port BRAM controllers, to low-level bare-metal ARM driver programming and physical HDMI transmission—the authors have created an exemplary demonstration of hardware/software co-design.

Key engineering takeaways and achievements of spGPU include:

1. **Spatial Multicore Scaling:** Dividing the display into 10 independent horizontal tiles (320×24) with dedicated compute cores and localized BRAM buffers eliminated memory contention and scaled geometric rendering throughput up to $10\times$.
2. **Intelligent Hardware Scheduling:** Bounding box pre-decoding ensures that instructions are dispatched exclusively to the cores whose tiles overlap the primitive, preventing idle cycle waste.
3. **Integrated Depth Buffering and Double Buffering:** Single-cycle 4-bit hardware Z-buffering and automatic background clearing during VSYNC-locked buffer swaps deliver robust 2.5D visual layering with zero screen tearing.
4. **Direct Physical Interface:** Driving high-speed differential TMDS HDMI video directly from the FPGA using native `OSERDESE2` primitives at 250 Mbps per lane without external video DACs.
5. **Deterministic 60 FPS Pacing:** A dual-interrupt architecture connecting the hardware performance analyzer (1 Hz) and the vertical sync generator (60 Hz) directly to the ARM GIC enables rock-solid real-time animation, rigid-body physics, and telemetry logging.

In summary, spGPU stands as a fully functional, highly optimized, and thoroughly verified custom graphics processing engine that demonstrates the full potential of FPGA technology in accelerating modern real-time visual workloads.